Proceedings of the 2023 International Conference on Machine Learning and Cybernetics, 9-11 July, 2023

IMPROVING THE ADVERSARIAL ROBUSTNESS OF DEEP NEURAL NETWORKS VIA EFFICIENT TWO-STAGE TRAINING

RAN WANG[1], JINGYU XIONG[1], HAOPENG KE[1], YUHENG JIA[2], DEBBY D. WANG[3]

[1]College of Mathematics and Statistics, Shenzhen University, Shenzhen 518060, China
[2]School of Computer Science and Engineering, Southeast University, Nanjing 211189, China
[3]School of Science and Technology, Hong Kong Metropolitan University, Kowloon, Hong Kong
E-MAIL: wangran@szu.edu.cn, 2070205051@email.szu.edu.cn, 2070205031@email.szu.edu.cn
yhjia@seu.edu.cn, dwang@hkmu.edu.hk

2023 International Conference on Machine Learning and Cybernetics (ICMLC) | 979-8-3503-0378-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICMLC58545.2023.10327974

Abstract:

Despite the widespread applications of deep neural networks (DNNs), the potential threats of adversarial examples remain considerable concerns. In this paper, we propose a new proactive defense method based on a two-stage training procedure in order to enhance the adversarial robustness of DNNs. The first stage intends to map the input samples into a feature embedding space with high separability, while the second stage fixes the feature generator and learns the parameters only for the last layer for classification. Different from most of the existing state-of-the-art methods, our method neither spends extensive costs to generate adversarial information, nor constructs a complex penalty function to force the model satisfying specific restrictions under subjective assumptions. Instead, our method cuts off the interactions between the parameter updating for the deep feature generator and the classifier, discovers the relationship between adversarial robustness and separability in embedding space, results in a better interpretability. Substantial experiments on various network structures deployed on benchmark data sets including MNIST, FASHION MNIST and CIFAR10 and their adversary from different attacks demonstrate the rationality of our method.

Keywords:

Adversarial robustness; Adversarial defense; Deep neural network; Two-stage training

1. Introduction

Deep neural networks (DNNs), as an important branch of modern machine learning, construct a network structure containing several latent layers to learn the representations and patterns of data. Despite the widespread applications of DNNs, there exists a quite annoying phenomenon called adversarial examples (AEs). AEs were introduced to DNNs by adding small disturbances to the original clean input image [11]. The disturbances are imperceptible to humans but can mislead the deployed DNNs to make wrong predictions.

Over the past decade, a lot of works have been proposed to protect DNNs from the attacks of AEs, namely adversarial defense (AD), and the model's capability to resist AEs is called adversarial robustness. The most representative AD method is adversarial training (AT) [7], which formulates a new supervisory loss function to introduce adversarial information into the training procedure. Later, some AT variants have been proposed by either utilizing adversarial information through a more complex training procedure, or imitating inherent effects of AT on parameters of input layer, latent layers and output layer. However, as observed in many works, AT could result in dramatic changes of the generated deep features and injure the accuracy on clean data. Another AD method is to add some artificially designed penalty terms to the loss function, such that the model could obey some properties beneficial for improving adversarial robustness [8, 9]. However, when the loss function is highly complex or non-convex, we can neither guarantee its convergence theoretically nor experimentally, wasting large-scale efforts.

In this paper, we believe that the adversarial robustness is strongly correlated to the data separability in feature embedding space. Intuitively, the higher separability among classes, the longer distance between each sample and the decision boundary, as such, an example needs a stronger perturbation to become an AE, which implies better adversarial robustness. Inspired by this, we propose a new proactive AD method based on a two-stage training procedure, the first stage aims to learn an

979-8-3503-0378-0/23/$31.00 ©2023 IEEE

embedding space with higher data separability geometrically, and the second stage uses the embedding features as input to learn a decision model. Namely, we deploy a softmax regression task in a fixed embedding space. For clarification, the main contributions of this work are listed below:

1. We propose a two-stage training framework to enhance the adversarial robustness of DNNs, where the two stages use different loss functions to train a feature generator and a classifier, respectively.
2. The feature generator employs ArcFace loss to ensure the separability of classes in embedding space [3], and the classifier employs SoftMax loss to enhance classification accuracy on input data.
3. Different from most state-of-the-art methods, our method involves neither complex penalty function nor adversarial information, which has gratifying efficiency.
4. We try to interpret the mechanism of the proposed method from a theoretical perspective, leading to better interpretability for both AE and AD.
5. We conduct substantial experiments to corroborate the competitiveness of the proposed work. Furthermore, ablation studies are conducted to investigate the effectiveness and efficiency.

2. Proposed Work

Given an arbitrarily fixed training set $\mathbb{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with N distinct examples and C classes, where $\mathbf{x}_i = (x_{i1}, x_{i2}, ..., x_{in}) \in \mathcal{X} \subseteq \mathbf{R}^n$ represents the i-th input sample and $y_i \in \mathcal{Y} = \{1, 2, ..., C\}$ represents the class label of $\mathbf{x}_i$. An ideal DNN model $F : \mathcal{X} \to \mathcal{Y}$ is supposed to perform well on the examples in $\mathbb{D}$. However, by adding imperceptible perturbations to examples in $\mathbb{D}$, AEs can be produced for model attack and seriously harm the performance of the well-trained DNN.

2.1 Decomposition Perspective of DNN

For a C-class classification task, we consider a DNN model F as a composite function $F(\mathbf{x}) = f \circ g(\mathbf{x})$, in which

- $g : \mathbf{x} \mapsto \mathbf{z} = g(\mathbf{x}), \mathbf{R}^n \to \mathbf{R}^m$ represents a deep feature generator for embedding space, where $\mathbf{z}$ is the m-dimensional embedding vector of input sample $\mathbf{x}$; and
- $f : \mathbf{z} \mapsto \hat{p} = f(\mathbf{z}), \mathbf{R}^m \to [0, 1]^C$ represents the last layer of DNN, where $\hat{p} = (\hat{p}_1, \hat{p}_2, ..., \hat{p}_C)$ is the score vector describing the possibilities of $\mathbf{x}$ belonging to each of the C classes.

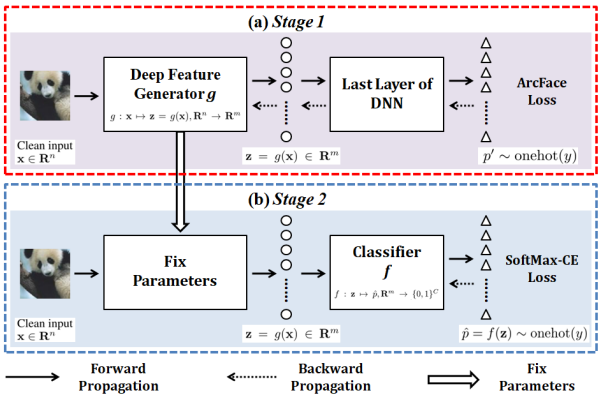


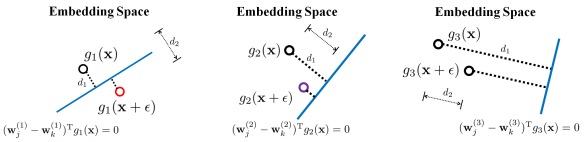
FIGURE 1. Framework of the proposed two-stage method, the two stages are supervised by different objective functions. (a) Stage 1: the parameters of feature generator g and the last layer of DNN (classification layer) are learned by ArcFace loss. (b) Stage 2: the parameters of g are fixed and the classifier f is learned again by Softmax cross-entropy loss.

DNN is a "black-box" model due to its opaque internal state and the process of generating deep features. During back-propagation (BP), the convergence of the parameters in the generator g and the classifier f are directly connected. In each iteration, g and f adapt to each other by calculating the gradient of the loss function, thus the classifier f is deployed in an alterable high-dimensional Euclidean subspace produced by g. In other word, a DNN model is the convergence result of a sequence of traditional classification tasks, which leads to difficulties for analyzing the internal changes of parameters. Motivated by these facts, we intend to artificially cut off the connections of the parameter updating for g and f, such that g could learn an embedding space with features beneficial for improving adversarial robustness, and f could learn the classifier in a fixed embedding space satisfying certain properties, i.e., a two-stage training framework. The framework of the proposed method is summarized in Figure 1.

2.2. Classifier f with Stronger Adversarial Robustness

Suppose that the deep feature generator g is fixed, we treat the last layer of DNN as a traditional classification task. SoftMax function is the most widely used activation function for the last layer.

Given that $\mathbf{z}_i = g(\mathbf{x}_i)$ is the $m \times 1$ embedding feature vector of input $\mathbf{x}_i$, $i = 1, \ldots, N$, $\mathbf{W}_{m \times C} = [\mathbf{w}_1, \ldots, \mathbf{w}_C]$ and $\mathbf{b} = (b_1, \ldots, b_C)$ are the parameters to be learned in the last layer, where $\mathbf{w}_j$ and b_j respectively represent the weight and bias parameters for the j-th output, $j = 1, \ldots, C$. The Soft-



(a) Situation 1 (b) Situation 2 (c) Situation 3

FIGURE 2. Three different attack situations: (a) attack succeeded, where $d_1 < d_2$; (b) attack almost succeeded, where d_1 is slightly larger than d_2; and (c) attack failed, where $d_1 \gg d_2$. If the generator g could map the input $\mathbf{x}$ farther away from the decision boundary in embedding space, the attack with limited ϵ is more likely to fail.

Max output is defined as:

$$f(\mathbf{z}_i) = (f_1(\mathbf{z}_i), \ldots, f_C(\mathbf{z}_i)), \tag{1}$$

where

$$f_j(\mathbf{z}_i) = \frac{e^{\mathbf{w}_j^{\mathrm{T}}\mathbf{z}_i + b_j}}{\sum_{k=1}^{C} e^{\mathbf{w}_k^{\mathrm{T}}\mathbf{z}_i + b_k}}, j = 1, \ldots, C. \tag{2}$$

We use the classic cross-entropy (CE) loss function to measure the difference between the real distribution of a label and its corresponding prediction:

$$\mathrm{L_{CE}} = -\sum_{i=1}^{N}\sum_{j=1}^{C} \mathbb{I}(y_i = j) \times \ln(f_j(\mathbf{z}_i)), \tag{3}$$

where $\mathbb{I}(\cdot)$ is an indicator function that equals to 1 if the inner condition holds and 0 otherwise. From classic analysis we could directly provide the decision boundary between class j and class k as follows:

$$(\mathbf{w}_j - \mathbf{w}_k)^{\mathrm{T}}\mathbf{z}_i = 0,\ j \neq k. \tag{4}$$

Note that for simplicity, we assume the bias parameter $b = 0$. Furthermore, $\left|(\mathbf{w}_j - \mathbf{w}_k)^{\mathrm{T}}\mathbf{z}_i\right|$ represents the geometric interval between embedding vector $\mathbf{z}_i$ and the decision hyperplane between class j and class k. For any correctly predicted input $\mathbf{x}_i$ under fixed adversarial strength ϵ, if $d_1 \gg d_2$, i.e.,

$$\left|(\mathbf{w}_j - \mathbf{w}_k)^{\mathrm{T}} g(\mathbf{x}_i)\right| \gg \left|(\mathbf{w}_j - \mathbf{w}_k)^{\mathrm{T}}(g(\mathbf{x}_i + \epsilon) - g(\mathbf{x}_i))\right|, \tag{5}$$

then it is almost certain that the adversarial robustness of DNN is appreciable, because the arbitrary perturbation ϵ added to $\mathbf{x}_i$ with fixed strength won't push the embedding vector $\mathbf{z}_i$ across the decision hyperplane, as shown in Figure 2.

Note that the above discussion does not involve the wrongly predicted data, such restriction infers that the inter-class discrepancy would increase. On the other hand, if the trained generator g could ensure a high degree of inter-class discrepancy and intra-class compactness with certain accuracy on clean training data, the adversarial robustness would reach a relatively high level.

2.3. Generator g with Higher Inter-class Discrepancy and Intra-class Compactness

As aforementioned, the objective of the deep feature generator g is to ensure that every sample can be far away from the decision boundary, such that a small perturbation is not enough to push it across the boundary to be an AE. This objective is very similar to that of the face recognition (FR) task, deploying ArcFace loss function [3] might be one of the most efficient state-of-the-art methods, which is defined as:

$$\mathrm{L_{ArcFace}} = -\sum_{i=1}^{N}\sum_{j=1}^{C} \mathbb{I}(y_i = j) \times \ln(f_j(\mathbf{z}_i)), \tag{6}$$

where $\mathbb{I}(\cdot)$ is the same indicator function as in (3), and

$$f_j(\mathbf{z}_i) = \frac{e^{s\cdot\cos(\theta_{j,i}+\mathfrak{m})}}{e^{s\cdot\cos(\theta_{j,i}+\mathfrak{m})} + \sum_{k\neq j} e^{s\cdot\cos\theta_{k,i}}}, \tag{7}$$

subject to

$$\theta_{j,i} = \arccos(\bar{\mathbf{w}}_j^{\mathrm{T}}\bar{\mathbf{z}}_i),\ \bar{\mathbf{w}}_j = \frac{\mathbf{w}_j}{||\mathbf{w}_j||},\ \bar{\mathbf{z}}_i = \frac{\mathbf{z}_i}{||\mathbf{z}_i||}, \tag{8}$$

where $\theta_{j,i} = \arccos(\bar{\mathbf{w}}_j^{\mathrm{T}}\bar{\mathbf{z}}_i)$ represents the included angle between the feature vector $\mathbf{z}_i$ and the weight vector $\mathbf{w}_j$ in embedding space. The main idea of ArcFace loss is to treat the dot product of $\mathbf{z}_i$ and $\mathbf{w}_j$ as the similarity of the two m-dimensional vectors. Then, by applying margin $\mathfrak{m}$ only to the angle between the embedding feature vector $\mathbf{z}_i$ and the weight vector of its ground truth label $\mathbf{w}_{y_i}$ (i.e., $\theta_{y_i,i}$), it could push $\mathbf{z}_i$ closer to $\mathbf{w}_{y_i}$ than to the weight vectors of other classes, thus enhancing the inter-class discrepancy and intra-class compactness.

In fact, various loss functions have been proposed recently to enhance the inter-class discrepancy and intra-class compactness [8,9,12]. However, many loss functions are composed of a modified SoftMax-CE loss and a well-crafted penalty term inferred from certain assumptions and constraints, hence the optimal decision boundary may converge to a very complex hypersurface. However, the ArcFace loss is directly generated from SoftMax function through a single penalty hyper-parameter $\mathfrak{m}$. In this case, we can expect the decision boundary to be hyperplane. Although this may injure the performance or increase the difficulty of convergence, it can provide convenience for analyzing adversarial robustness and a high level of interpretability in the second stage. Besides, it implies clear geometrical

Algorithm 1 The proposed two-stage AD method.

Input: Training set $\mathbb{D}$, feature scale s, margin parameter $\mathfrak{m}$, DNN structure and maximum number of iterations N.
Parameters: Weight parameters $\mathbb{W}_1$ of generator g and weight parameter $\mathbf{W}_2$ of classifier f.
Output: Class-wise score $\hat{p}$

Initialize random parameters $\mathbb{W}_1$ for g and $\mathbf{W}_2$ for f;
Stage 1:
for $n = 1$ to N **do**
  Calculate ArcFace loss $\mathrm{L}_{\mathrm{ArcFace}}$ based on (6)$\sim$(7);
  Update $\mathbb{W}_1$ and $\mathbf{W}_2$ based on the gradient of $\mathrm{L}_{\mathrm{ArcFace}}$ through BP;
end for
Fix $\mathbb{W}_1$;
Stage 2:
for $n = 1$ to N **do**
  Calculate CE loss L_{CE} based on (2)$\sim$(3);
  Update $\mathbf{W}_2$ based on the gradient of L_{CE} through BP;
end for
Calculate $\hat{p} = f_{\mathbf{W}_2} \circ g_{\mathbb{W}_1}(\mathbf{x})$ for each input $\mathbf{x}$;
return $\hat{p}$.

interpretation related to the inference of classifier. Finally, the proposed two-stage AD method is described in Algorithm 1.

3. Experiment

In this section, we compare the proposed two-stage defense method with ST and AT on real-world image data sets. We also report the results of some recent state-of-the-art defense methods for a more comprehensive comparison. The adversarial robustness of a DNN model is evaluated by its accuracy on adversarial test data generated through typical attacks.

3.1. Data Sets and Environment

We deploy the methods on three benchmark image data sets. **MNIST**: This data set includes 70,000 28×28 grayscale images with 60,000 for training and 10,000 for testing. Each image is a $0 \sim 9$ handwritten digit. **FASHION MNIST**: This data set includes 70,000 28×28 grayscale images with 60,000 for training and 10,000 for testing. Each image is sampled from ten types of fashion goods. **CIFAR10**: This data set includes 60,000 32×32 color images with 50,000 for training and 10,000 for testing. All images are labeled solely in one of the following ten class: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. For software we employ PyTorch and AdverTorch library under Linux operating system. For hardware we use eight pieces of 2080ti GPU with a maximum of 128Gb memory. All codes involved in this paper can be downloaded via the link[1].

3.2. Experimental Settings

Two different DNN structures are employed, i.e., LeNet-5 and ResNet-18. In all the experiments, Adam optimizer is utilized, where the random seed is consistently set as 6666. For both ST and AT, the learning rate is consistently set as 0.001, and the number of epoches is listed in Table 3. For the proposed method, the number of epoches in the first stage, the number of epoches in the second stage, the learning rate, scale s, and margin $\mathfrak{m}$ are set as $(200, 50, 0.01, 30, 0.4)$ for MNIST and $(500, 50, 0.001, 60, 0.4)$ for FASHION MNIST with LeNet-5; and set as $(750, 50, 0.0003, 60, 0.5)$ for MNIST, $(1500, 50, 0.0003, 60, 0.5)$ for FASHION MNIST, and $(1500, 50, 0.001, 30, 0.5)$ for CIFAR10 with ResNet-18. Note that we did not apply LeNet-5 on CIFAR10 due to its poor performance on this data set. Furthermore, based on some trial experiments, it is discovered that the ArcFace loss is difficult to converge on FASHION MNIST with LeNet-5 and CIFAR10 with ResNet-18, thus in these two experiments, we replace ArcFace loss with CosFace loss [13] for the first stage.

3.3. Adversarial Attacks

We choose five typical adversarial attacks, i.e., fast gradient sign method (FGSM) [5], projected gradient attack (PGD) [7], basic iteration method (BIM) [6], boosting adversarial attack (MIM) [4], and Carlini and Wagne's Attack (C&W) [1]. We follow [10] for the parameter settings of the attacks. **MNIST/FASHION MNIST**: The adversarial strength ϵ is 0.3, and the step size of iterative attack is 0.01. The number of iterations is set as 10, 40, 40 and 1000 for BIM, MIM, PGD and CW, respectively. For CW, the confidence is 0, learning rate is 0.01, and initial constant is 0.001. **CIFAR10**: The adversarial strength ϵ is 8/255, and the step size of iterative attack is 1/255. Other parameters are same as mentioned above.

3.4. Experimental Results

Table 1 reports the testing accuracy of the DNN model with LeNet-5 structure trained by different methods on the clean test examples and the adversarial test examples under the five attack methods for data sets MNIST and FASHION MNIST. Note that we did not report the result of this network structure on CIFAR10 due to its poor performance on this data set (we guess this is because LeNet-5 is a simple DNN model with few

[1] https://github.com/K-Tanjirou/adversarial-example-project

TABLE 1. Testing accuracy of different defense methods under different attacks with LeNet-5 structure.

Method	Clean	FGSM	PGD	BIM	MIM	C&W
MNIST						
ST	99.19%	27.81%	1.12%	1.35%	1.61%	0.40%
AT-PGD	96.83%	87.51%	82.48%	80.50%	81.81%	2.50%
Chen's	**99.22%**	74.85%	8.32%	66.30%	64.02%	1.34%
Ours	99.06%	**99.06%**	**98.76%**	**99.06%**	**99.06%**	**99.06%**
FASHION						
ST	**91.29%**	17.00%	4.02%	5.43%	5.71%	6.51%
AT-PGD	67.58%	68.58%	**67.94%**	68.31%	60.38%	**33.79%**
Ours	89.77%	**79.16%**	57.06%	**79.36%**	**79.39%**	5.19%

Note: AT-PGD means AT with AEs generated by PGD attack, and the result of Chen's method is the original result from [2].

TABLE 2. Testing accuracy of different defense methods under different attacks with ResNet-18 structure.

Method	Clean	FGSM	PGD	BIM	MIM	C&W
MNIST						
ST	**99.56%**	14.81%	0.17%	0.37%	0.36%	0.39%
AT-PGD	99.33%	**97.35%**	**96.18%**	94.87%	95.52%	**0.60%**
Ours	99.53%	95.04%	82.33%	**97.16%**	**97.12%**	0.43%
FASHION						
ST	93.76%	15.11%	2.26%	4.08%	4.38%	4.62%
AT-PGD	83.26%	**81.30%**	**79.24%**	76.38%	71.00%	**74.20%**
Ours	**93.92%**	77.40%	15.63%	**88.56%**	**88.80%**	4.50%
CIFAR10						
ST	91.99%	16.12%	5.07%	6.39%	5.25%	5.19%
AT-PGD	76.69%	61.10%	55.18%	57.11%	55.79%	**12.34%**
Chen's	**94.56%**	51.71%	7.30%	25.30%	10.19%	0.14%
Pang's	92.70%	——	24.80%	——	——	0.17%
Ours	91.86%	**64.29%**	**60.79%**	**63.84%**	**63.72%**	5.22%

Note: AT-PGD means AT with AEs generated by PGD attack, the result of Chen's method is the original result from [2], and the result of Pang's method is the original result from [9]. Note that Pang's work uses ResNet-32 instead of ResNet-18.

layers, it is hard to fit a complex data set). It can be clearly seen from Table 1 that for both data sets, the proposed two-stage method achieves higher accuracy on adversarial test examples compared with ST, and achieves higher accuracy on clean test examples compared with AT. Besides, although AT outperforms the proposed method on adversarial test examples under PGD and C&W attacks for FASHION MNIST, it's accuracy on the clean test examples has deteriorated a lot, which clearly shows that AT could result in dramatic changes of the generated deep features and injure the accuracy on clean data. However, the proposed method demonstrates a very strong adversarial robustness while maintaining good performance on clean data. This may due to the fact that it did not introduce any adversarial information during the training process, thereby did not harm the deep feature generation.

Table 2 reports the testing accuracy of the DNN model with ResNet-18 structure on the clean test examples and the adversarial test examples for data sets MNIST, FASHION MNIST and CIFAR10. Similarly, compared with ST, the proposed two-stage method demonstrates a very strong adversarial robustness while maintaining good performance on clean data. The deterioration of its accuracy on clean test examples is quite limited. However, it fails to outperform AT under FGSM, PGD and C&W attacks in some cases. Since ResNet-18 is a much more complex network structure compared with LeNet-5, it has a stronger capability to fit various data sets. Despite this, the proposed method still outperforms others in many cases, especially on data set CIFAR10.

TABLE 3. Average training time per epoch of different defense methods on different data sets (seconds).

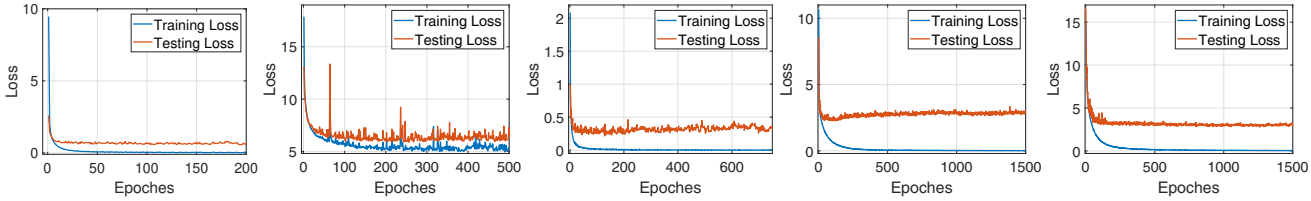
Data Set	MNIST		FASHION MNIST		CIFAR10
	LeNet-5	ResNet-18	LeNet-5	ResNet-18	ResNet-18
ST	8.02	28.31	12.95	67.41	56.36
	(50)	(50)	(150)	(50)	(200)
AT-PGD	43.12	785.37	51.36	798.29	1000.93
	(30)	(30)	(50)	(50)	(100)
Ours	7.79	30.75	11.90	43.71	35.92
	(200+50)	(750+50)	(500+50)	(1500+50)	(1500+50)

Note: AT-PGD means AT with AEs generated by PGD attack, the integer in parentheses means the total number of training epochs for each experiment.

Finally, Table 3 reports the average training time per epoch of different methods on different data sets, and the total number of epoches in each experiment. Clearly, the training cost per epoch in the proposed method is similar to or even lower than that of ST, and these two methods are much faster than AT. Note that for the proposed method, the total number of epoches includes two training stages. Especially, in order to sufficiently pull apart the classes in feature embedding space and guarantee a high level of separability, we set a very large number of epoches in the first training stage. In fact, as shown in Figure 3, the model in the first stage can converge in a much earlier iteration. Furthermore, as shown in Figure 4, we can see that in the second training stage, the parameters can be learned at the beginning and remain stable after. This is because the samples in the embedding space become highly and linearly separable, the separating boundaries among them are very easy to learn. These two investigations tell that by setting an appropriate smaller number of epoches in the two stages, the proposed method can achieve a very strong adversarial robustness with much higher efficiency than the state-of-the-art AT methods.

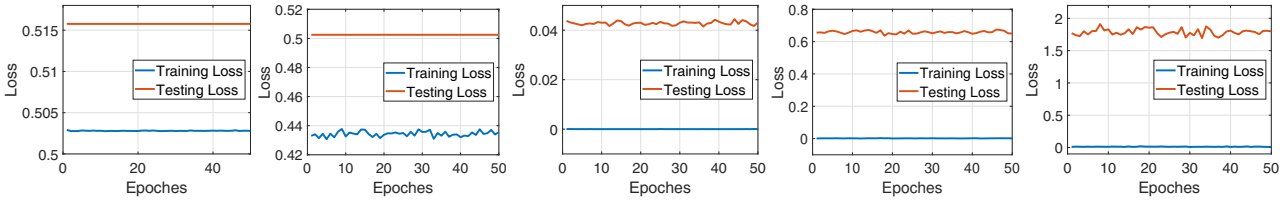
4. Conclusions

We propose a new proactive defense method based on a two-stage training procedure without extra crafted penalty function and adversarial information. We suggest a different elaboration on DNN model and further discuss the rationality of our method. Substantial experiments demonstrate that our method could improve the adversarial robustness of DNN model compared with ST, and at the same time maintain a good perfor-



(a) LeNet-5: MNIST (b) LeNet-5: FASHION MNIST (c) ResNet-18: MNIST (d) ResNet-18: FASHION MNIST (e) ResNet-18: CIFAR10

FIGURE 3. Convergence of loss in the first training stage of the proposed method.



(a) LeNet-5: MNIST (b) LeNet-5: FASHION MNIST (c) ResNet-18: MNIST (d) ResNet-18: FASHION MNIST (e) ResNet-18: CIFAR10

FIGURE 4. Change of loss in the second training stage of the proposed method.

mance on clean data compared with AT. This can be encouraging, as there are potentials for designing new loss functions under the proposed procedure, or combining the proposed procedure with state-of-the-art defense methods to further improve the adversarial robustness of DNN models.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (Grants 62176160 and 62203304), in part by the Guangdong Basic and Applied Basic Research Foundation (Grant 2022A1515010791), and in part by the Hong Kong Metropolitan University (Project 2022/23 S&T - R5112).

References

[1] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy*, pages 39–57. IEEE, 2017.

[2] Sihong Chen, haojing Shen, Ran Wang, and Xizhao Wang. Relationship between prediction uncertainty and adversarial robustness. *Journal of Software*, 33(2):524–538, 2022.

[3] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *CVPR*, pages 4690–4699, 2019.

[4] Yinpeng Dong, Fangzhou Liao, Tianyu Pang, Hang Su, Jun Zhu, Xiaolin Hu, and Jianguo Li. Boosting adversarial attacks with momentum. In *CVPR*, pages 9185–9193, 2018.

[5] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *ICLR*, 2015.

[6] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial machine learning at scale. In *ICLR*, 2017.

[7] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *ICML*, 2018.

[8] Aamir Mustafa, Salman Khan, Munawar Hayat, Roland Goecke, Jianbing Shen, and Ling Shao. Adversarial defense by restricting the hidden space of deep neural networks. In *ICCV*, pages 3384–3393, 2019.

[9] Tianyu Pang, Kun Xu, Yinpeng Dong, Chao Du, Ning Chen, and Jun Zhu. Rethinking softmax cross-entropy loss for adversarial robustness. In *ICLR*, 2020.

[10] Nicolas Papernot, Patrick McDaniel, Somesh Jha, Matt Fredrikson, Z Berkay Celik, and Ananthram Swami. The limitations of deep learning in adversarial settings. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 372–387. IEEE, 2016.

[11] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *ICLR*, 2014.

[12] Weitao Wan, Yuanyi Zhong, Tianpeng Li, and Jiansheng Chen. Rethinking feature distribution for loss functions in image classification. In *CVPR*, pages 9117–9126, 2018.

[13] Hao Wang, Yitong Wang, Zheng Zhou, Xing Ji, Dihong Gong, Jingchao Zhou, Zhifeng Li, and Wei Liu. Cosface: Large margin cosine loss for deep face recognition. In *CVPR*, pages 5265–5274, 2018.